SSH Setup

Big Data Algorithms — setup guide

After you register with the moriah cluster, and do lecture 4, continue with this setup document.

Logging in with keys and `~/.ssh/config`

Making your life logging in easier

Typing a password each time gets tedious. A **key pair** replaces it: you create a private key that stays on your computer and a public key that you give to the server:

```
#ssh-keygen -t ed25519    # make the key pair, once (you've done this already for github)
ssh-copy-id user@server  # install your public key on the server
ssh user@server          # now logs in without a password
```

`~/.ssh/` configuration files

The folder `~/.ssh/` in your home folder holds the configuration files for `ssh`: your keys, your settings, and the servers you know. Take a look:

```
ls ~/.ssh/
```

Yours will show some of these:

File	What it is	Share it?
`id_ed25519`	your default **private key**, made by `ssh-keygen -t ed25519`	**never**
`id_ed25519.pub`	its **public key**, the line `ssh-copy-id` installs on servers	yes - grants access

File	What it is	Share it?
`myserver_key`	an extra **private key**, made for one particular server	**never**
`myserver_key.pub`	its public key	yes - grants access
`myserver_key.ppk`	the same private key in the format of PuTTY, a Windows `ssh` program	**never**
`config`	your `ssh` settings, such as short names for servers (below)	not secret, but private
`known_hosts`	the fingerprint of every server you have connected to	not secret
`known_hosts.old`	a backup made when an entry is removed from `known_hosts`	not secret

`ssh` uses `id_ed25519` automatically. Any other key you name with `-i` or with an `IdentityFile` line in `config`. Some people make a separate key pair for each server, so that they can withdraw one without touching the others.

`known_hosts` protects you from impostors. The first time you connect to a server, `ssh` asks `Are you sure you want to continue connecting?`. If you answer `yes`, it saves the server's fingerprint here. On every later login it checks that the server shows the same fingerprint. If not, it stops with a loud `REMOTE HOST IDENTIFICATION HAS CHANGED` warning, because someone may be pretending to be the server. (The usual innocent cause is a server that was reinstalled. Then `ssh-keygen -R server` removes the old entry and saves the backup `known_hosts.old`.)

Public keys are like deadbolt locks (and the private key is the key). You can share the public key. Anyone is allowed to install your deadbolt on any door allowing you access.

`~/.ssh/config`: give the server a short name

We make our lives easier in two steps. First, the Moriah command is long to type every time. Put the details in the file `~/.ssh/config` on your own computer (create it if it is not there), using your own login in place of `kapelner`:

```
cat >> ~/.ssh/config <<'EOF'
Host moriah
    HostName moriah-gw.cs.huji.ac.il
    User kapelner
    ProxyJump kapelner@bava.cs.huji.ac.il
EOF
cat ~/.ssh/config # check it - you should see Host github.com ...<3 tabbed lines>... Host mo
```

`Host` is the short name you choose. `HostName` is the real address, `User` is your login, and `ProxyJump` is the same jump as `-J`. Now this is all you type:

```
ssh moriah
```

The short name works everywhere `ssh` is used underneath, including `ssh-copy-id`, and `scp` and `rsync` from Lecture 4. To log in to Moriah without its password, install your key there:

```
ssh-copy-id moriah
```

You still type the one-time password (OTP) for `bava`. This command will ask for at least three OTPs, because it makes three separate connections through `bava`: one to read which version of `ssh` Moriah runs, one to check whether your key is already installed there, and one to install it (this last one also asks for your Moriah password). Make sure you wait for a new OTP code for each prompt.

Do not run `ssh-copy-id` on `bava` itself: your account there only forwards connections and has no shell, so it stops with `This account is currently not available` and installs nothing.

Now we make our lives even easier by asking the `bava` to persist for the next four hours:

```
cat >> ~/.ssh/config <<'EOF'
Host bava.cs.huji.ac.il
    User kapelner
    ControlMaster auto
    ControlPath ~/.ssh/cm-%r@%h-%p
    ControlPersist 4h
    ServerAliveInterval 30
    ServerAliveCountMax 3
EOF
```

`ControlMaster` keeps one connection to `bava` open, and every later `ssh moriah` (and `scp`, `rsync`) reuses it through the socket file named by `ControlPath`, so no new OTP is needed. `ServerAlive...` makes `ssh` check every 30 seconds that the connection is still alive and close it after 3 missed answers, instead of leaving behind a dead connection (see the troubleshooting section below).

and let's enter the OTP once

```
ssh moriah
exit
```

and try again to ssh in:

```
ssh moriah
```

And voila! No OTP and no password required!!!

Troubleshooting: `ssh moriah` hangs

Sometimes `ssh moriah` just sits there and prints nothing. Usually Moriah is fine, and the problem is on your own computer. To see where it gets stuck, run it with `-v` (verbose):

```
ssh -v moriah
```

If the last line is

```
debug1: auto-mux: Trying existing master at '/home/kapelner/.ssh/cm-kapelner@bava.cs.huji.ac
22'
```

then the shared connection to `bava` has died (typically your laptop slept, or your Wi-Fi or VPN changed), but its socket file is still there, so `ssh` waits for an answer that never comes. Close it and delete the socket, then log in again (with a fresh OTP):

```
ssh -O exit bava.cs.huji.ac.il
rm -f ~/.ssh/cm-*@bava.cs.huji.ac.il-22
ssh moriah
```

The `ServerAliveInterval` and `ServerAliveCountMax` lines above make this rare, because `ssh` then notices a dead connection within about 90 seconds and cleans it up by itself.

If instead `-v` stops at `Connecting to bava.cs.huji.ac.il`, the problem is the network: check your internet connection (and the university VPN if you are off campus) and try again in a few minutes.